Cours 6

Forge, mots de passe et secrets

1re année géomatique
2027-2028

Objectifs de la séance

Cours 3	→	Cours 5	→	Cours 6
permettre à une autre personne d'utiliser ce code		installer ce code et ce dont il dépend		partager ce code et travailler à plusieurs

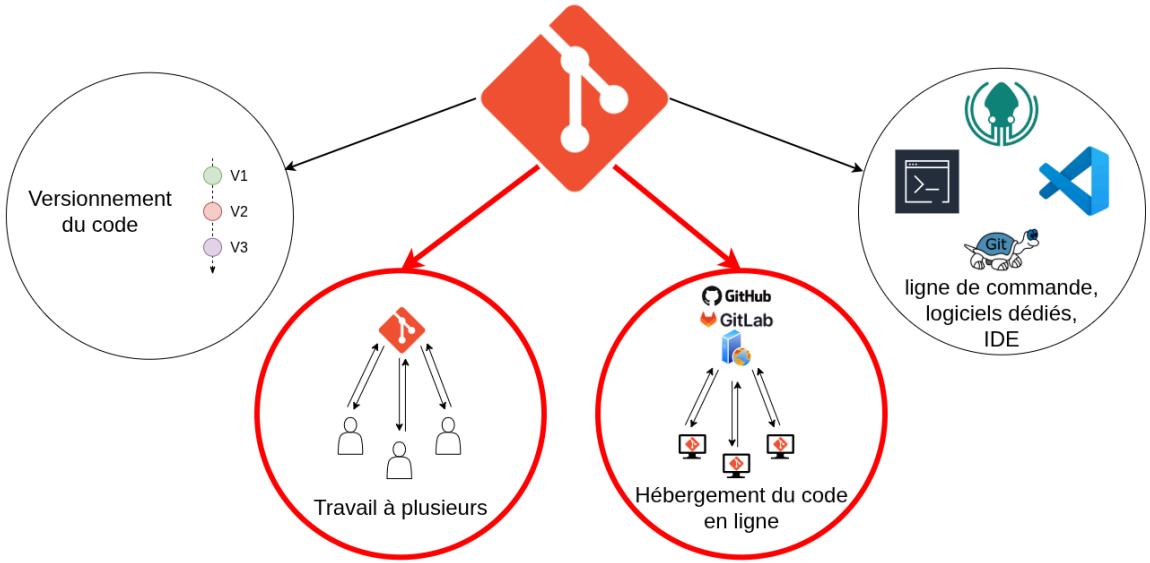
Synchroniser un dépôt avec une forge, et travailler à plusieurs sur ce dépôt.
Protéger l'accès au dépôt : authentification, droits, secrets.

Partager son code sur une forge

Les usages de git, et les commandes qui synchronisent un dépôt avec une forge.

Utilisations de git

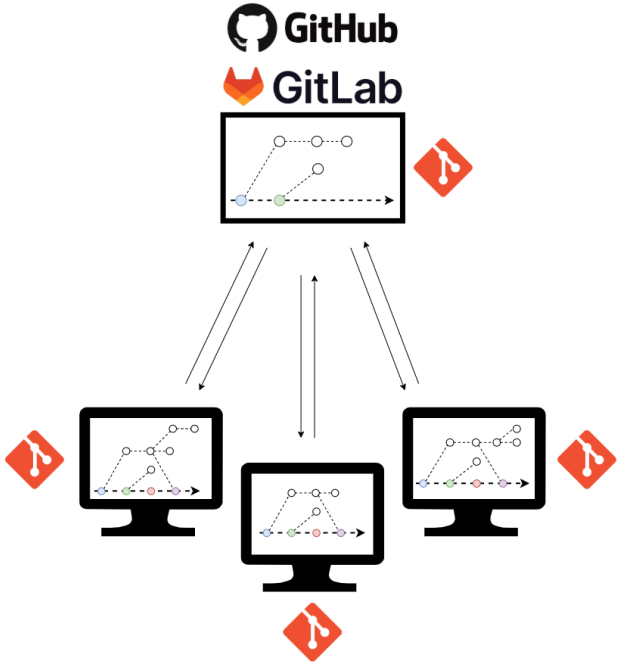
Le cours 2 versionne le code sur un poste. Ce cours porte sur le travail à plusieurs et sur l'hébergement du code en ligne.



Une forge, comme GitHub ou GitLab, héberge des dépôts git. Partie adaptée du cours de Florent Geniet, 14 octobre 2026.

Système décentralisé

git est décentralisé : chacun a sa propre copie du projet, avec son historique. La forge est une copie de plus, que l'équipe prend pour référence.



Synchronisation des dépôts : git clone

`git clone` récupère un dépôt depuis un serveur distant.



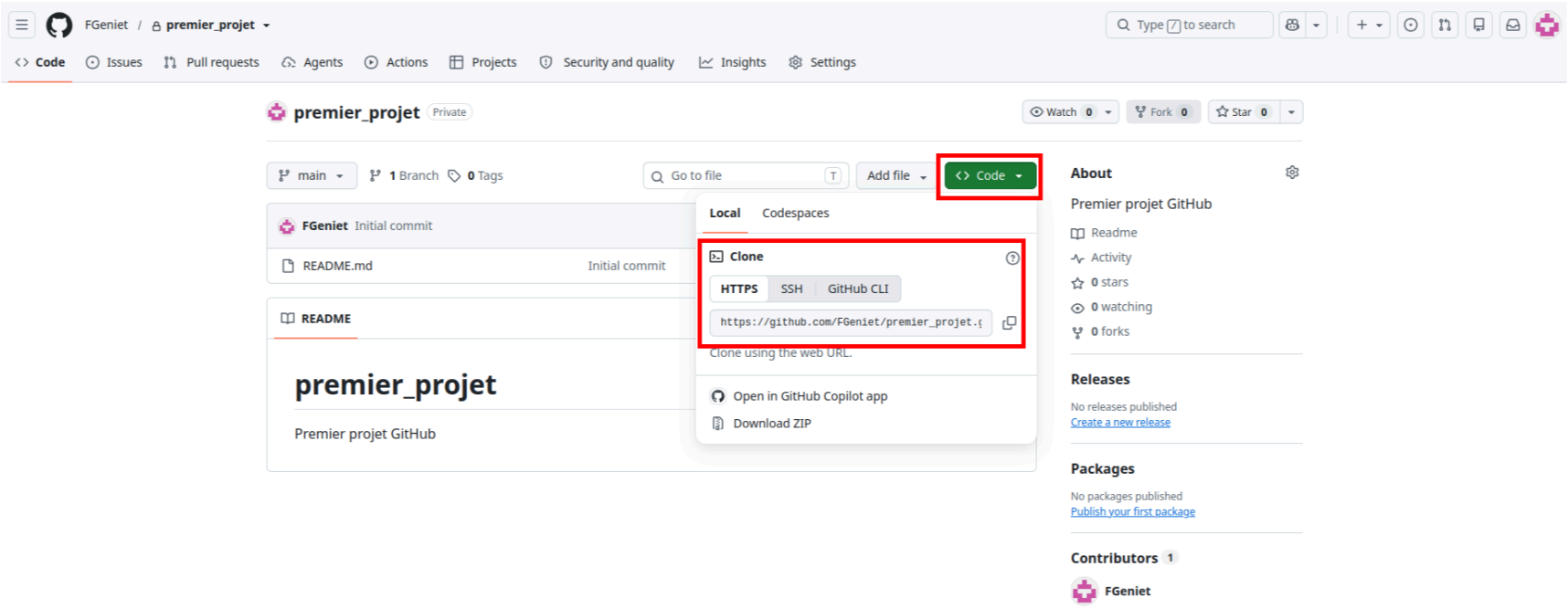
Git Bash

```
$ git clone <url du projet>
```

`<url du projet>` : à remplacer par l'adresse du dépôt, sans les chevrons.

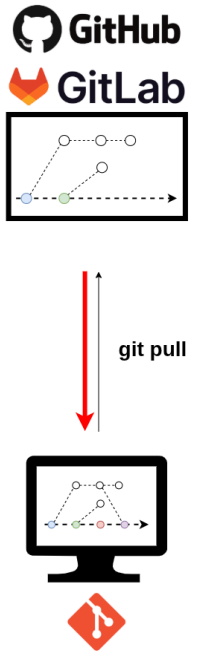
Synchronisation des dépôts : adresse du projet

L'adresse se copie sur la page du dépôt : bouton Code, puis onglet HTTPS ou SSH.



Synchronisation des dépôts : git pull

`git pull` récupère les commits du dépôt distant, et met à jour son propre dépôt.



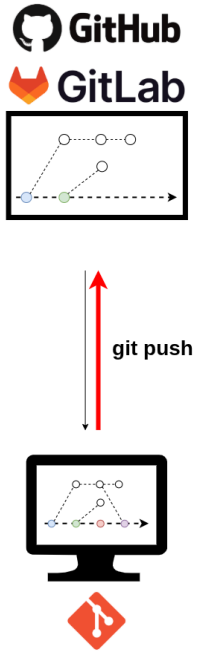
Git Bash

```
$ git pull
```

Si les deux dépôts ont des commits différents, `git pull` les fusionne, comme `git merge` au cours 2.

Synchronisation des dépôts : git push

`git push` envoie ses commits au dépôt distant, et le met à jour.



Git Bash

```
$ git push
```

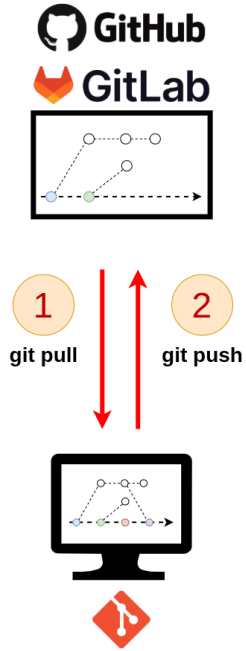
Commit local et push sur le réseau

Le commit écrit sur le disque de l'ordinateur, le push traverse le réseau. Git est fait pour qu'on puisse travailler sans réseau et pousser plus tard.



Synchronisation des dépôts : pull avant push

Un `pull` précède un `push` : le dépôt distant refuse un `push` s'il contient des commits que le poste n'a pas.



Git Bash

```
$ git pull
$ git push
```

Consommation électrique des services en ligne

Un fichier gardé en ligne consomme même quand personne ne le lit.

Quoi	Électricité	Équivalent
une question à un assistant d’IA	0,3 à 2 Wh	10 s à 1 min de télévision
la même, modèle « qui raisonne »	7 à 33 Wh	4 à 20 min de télévision
une heure de vidéo en streaming	80 Wh	une ampoule LED de 10 W, 8 h
garder 1 To en ligne pendant un an	40 à 150 kWh	1 à 3 disques toujours allumés
le même To dans un tiroir	0	
centres de données du monde, 2024	415 TWh	1,5 % du mondial, la France entière

IA : Jegham et al. 2025, serveurs seuls ; IEA 2020 et 2025 ; disque : 5 W × 8 760 h × 1 à 3 copies ; France : RTE 2024 ; télévision : 100 W.

Coût d’un service en ligne

Le coût suit le nombre de machines qui tournent, et de personnes qui les font tourner.

Service	Ce qui tourne	Par an
un site personnel, pages statiques	une part d’un serveur partagé, ou GitHub Pages	0 à 60 €
OpenStreetMap, la carte du monde	deux salles de serveurs, Amsterdam et Dublin	100 000 à 170 000 €
Wikipédia	plusieurs centres de données	3,4 M$ d’hébergement
Meta : Facebook, Instagram, WhatsApp	des dizaines de centres de données	72 milliards de $ d’investissement

Nom de domaine 10 € par an, hébergement partagé 3 à 5 € par mois ; OSMF, plan 2023, diffusion des tuiles offerte ; Wikimedia, exercice 2024-2025 ; Meta, résultats 2025.

Dépôt en ligne : authentification, droits, piratage

Une forge doit savoir qui écrit dans un dépôt. Un compte volé donne accès à tout ce que ce compte a le droit de faire.

Authentification

prouver qui l'on est : jeton, clé SSH, deuxième facteur

Droits

ce que chaque compte peut faire sur un dépôt : lire, écrire, administrer

Piratage

avec un compte volé : lire un dépôt privé, pousser du code malveillant, effacer un dépôt

Git Bash

```
$ git clone https://eleve-exemple:motdepasse@github.com/eleve-exemple/projet-prive.git
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/eleve-exemple/projet-prive.git/'
```

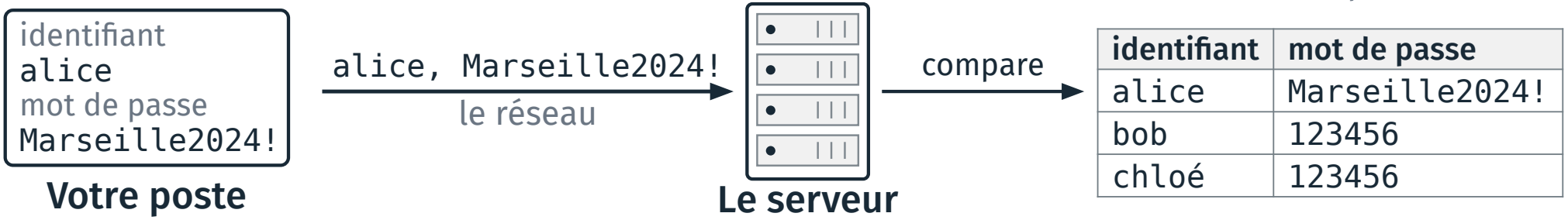
GitHub refuse le mot de passe du compte pour git depuis le 13 août 2021. En septembre 2025, le logiciel malveillant Shai-Hulud a touché plus de 500 paquets JavaScript avec les identifiants de leurs développeurs (CISA, agence américaine).

Mots de passe et clés SSH

Le mot de passe, les quatre attaques possibles, leurs parades, puis la paire de clés et le token qui le remplacent.

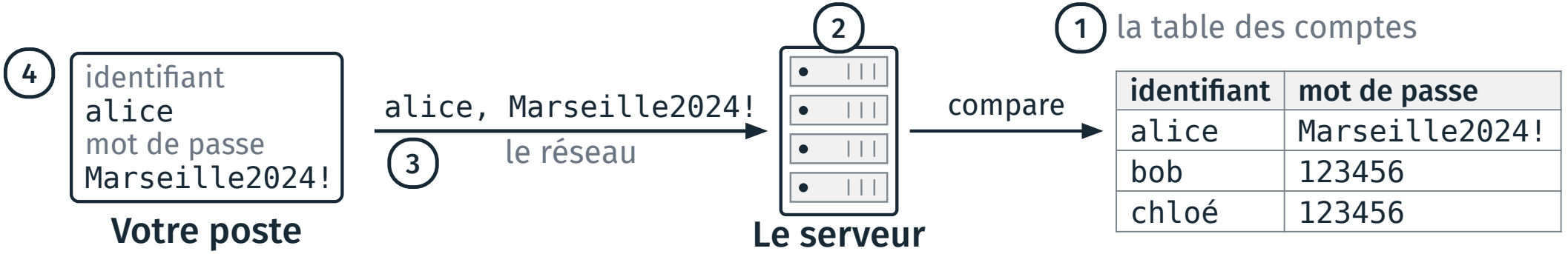
Connexion à un site par mot de passe

Le site garde chaque compte dans une table : l'identifiant et le mot de passe. À la connexion, le serveur compare le mot de passe reçu avec celui de la table.



Quatre attaques contre un mot de passe

Le mot de passe se lit à trois endroits : la table, le réseau, le poste. Il peut aussi être deviné.



1. lire la table : une fuite de données du site
2. deviner : essayer des mots de passe au formulaire
3. intercepter : lire le réseau s'il n'est pas chiffré
4. le prendre sur le poste : hameçonnage, logiciel espion

22 % des intrusions commencent par un identifiant et un mot de passe volés (Verizon, **Data Breach Investigations Report** 2025). Le site haveibeenpwned.com indique les fuites où figure une adresse de courriel.

Fonction de hachage et empreinte

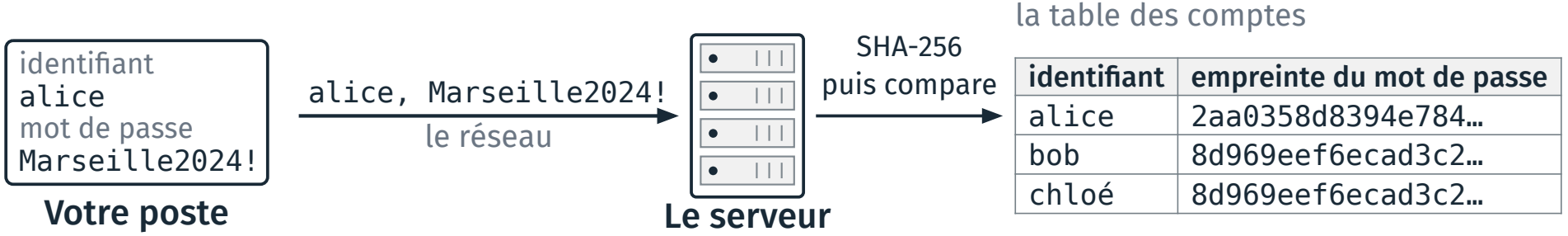
Une fonction de hachage répond à la question : ces deux contenus sont-ils identiques ? Elle calcule pour chaque contenu une empreinte de taille fixe, et on compare les empreintes.

```
bonjour  2cb4b1431b84ec15d35ed83bb927e27e8967d75f4bcd9cc4b25c8d879ae23e18
Bonjour  9172e8eec99f144f72eca9a568759580edadb2cfd154857f07e657569493bc44
```

Propriété de SHA-256	Sur l'exemple
le même contenu donne toujours la même empreinte	`bonjour` donne `2cb4b143…` partout
une lettre de différence donne une empreinte sans rapport	`Bonjour` donne `9172e8ee…`
l'empreinte a toujours 64 caractères	pour un mot, pour un fichier de 1 Go
aucun calcul ne retrouve le contenu à partir de l'empreinte	`2cb4b143…` ne permet pas de calculer `bonjour`

Table des comptes avec empreintes

Le serveur garde l'empreinte du mot de passe. À la connexion, il calcule l'empreinte du mot de passe reçu et la compare à celle de la table.



Avec la table volée	Résultat
taper `2aa0358d…` dans le formulaire	le serveur calcule l'empreinte de `2aa0358d…` : `1ea04f0b…`, différente de celle de la table
calculer l'empreinte de mots de passe candidats et comparer	chez soi, sans limite d'essais : la devinette, attaque 2

Empreinte avec sel

Sans sel, deux comptes qui ont le même mot de passe ont la même empreinte. Le serveur tire au hasard un sel pour chaque compte, et calcule l'empreinte du sel suivi du mot de passe.

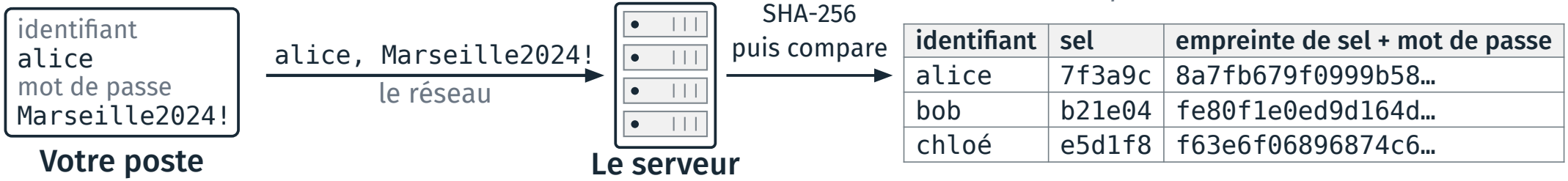


Table volée, sels compris	Sans sel	Avec sel
calculs pour essayer `123456`	1, qui trouve bob et chloé	3, un par compte
liste publique d'empreintes déjà calculées	donne `123456` pour `8d969eef…`	ne sert pas : l'empreinte dépend du sel

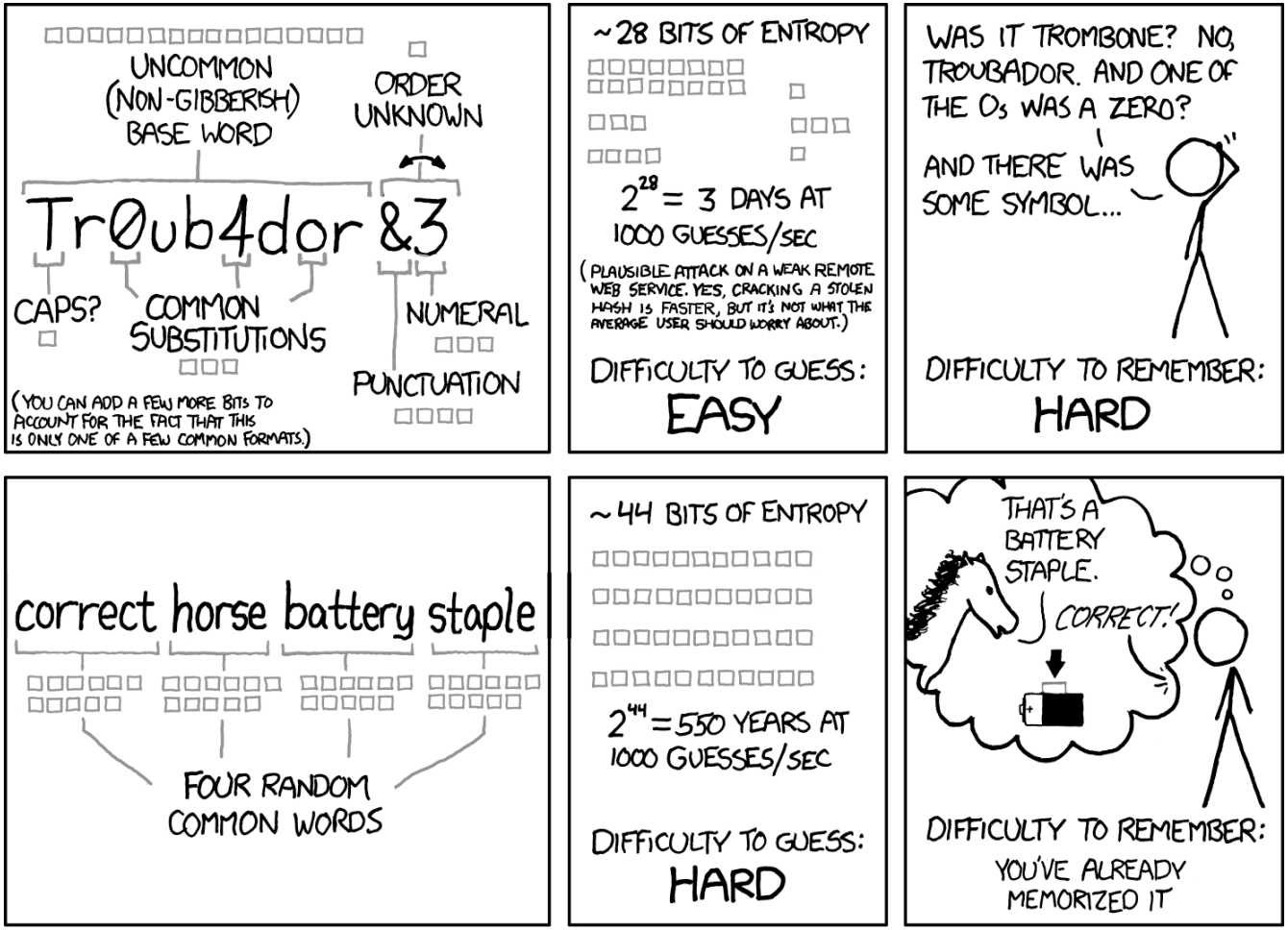
ANSSI 2021, § 4.6 : un sel tiré au hasard pour chaque compte, de 128 bits au moins.

Empreinte à calcul lent

Avec le sel, le voleur peut encore essayer des mots de passe, compte par compte. Une fonction lente, comme bcrypt, réduit le nombre d'essais par seconde.

Fonction	Essais par seconde	Usage	8 caractères tirés au hasard
SHA-256	22 milliards	vérifier un fichier, identifier un commit	3,5 jours
bcrypt (coût 5)	184 000	garder un mot de passe : le calcul est lent exprès	1 100 ans

Une carte graphique RTX 4090, hashcat 6.2.6 (Chick3nman, 2022) ; 95^8 combinaisons. ANSSI 2021, § 4.6 : une fonction lente pour garder les mots de passe.



Randall Munroe, xkcd n° 936, **Password Strength**, 2011, CC BY-NC 2.5.

L’entropie d’un mot de passe

n bits d’entropie font 2^n mots de passe possibles, et autant d’essais. L’entropie compte les choix faits au hasard.

Dans le dessin	Le compte
`Tr0ub4dor&3`	un mot peu courant (16 bits), une majuscule, des substitutions connues, un chiffre, un signe : 28 bits
`correct horse battery staple`	4 mots tirés au hasard parmi 2 048 : 4 × 11 = 44 bits
1 000 essais par seconde	des essais au formulaire ; sur une table bcrypt volée, 44 bits tombent en 4 mois

La CNIL (2022) demande 80 bits pour un mot de passe seul : 12 caractères de quatre classes, 14 de trois, ou 7 mots tirés au hasard.

Temps pour deviner un mot de passe

Avec une table d'empreintes volée, le voleur essaie les mots de passe dans l'ordre le plus probable, sur seize cartes graphiques.

Mot de passe	Comment il est trouvé	Entropie	Temps
`123456`	il est dans les listes de fuites	0 bit	instantané
`Marseille2024!`	un nom de ville, une année, un signe à la fin : des règles connues des programmes	≈ 30 bits	secondes à heures
`k7#Qp2vL`, 8 caractères tirés au hasard	toutes les combinaisons : 95^8	53 bits	132 ans
`lune orange pont carte vélo sel pluie`	toutes les combinaisons : $7\,776^7$	90 bits	hors de portée

Hive Systems, table 2026 : empreintes bcrypt de coût 10, seize RTX 5090. Au formulaire, le serveur limite les essais ; ces temps sont ceux d'une table volée.

Hameçonnage

Attaque 4 : le mot de passe est donné par son propriétaire, sur une page qui imite le site.

○○○ Boîte de réception

De : Service informatique <support@securite-compte.net> 1
Objet : Votre compte sera suspendu dans 24 h 2

Bonjour,

Suite à une mise à jour de nos systèmes, vous devez confirmer votre identité sous 24 heures, faute de quoi votre accès sera suspendu.

https://ecole.fr/compte 3 (le lien pointe vers `http://ecole-fr.verif-compte.net/…`)

Merci de votre compréhension.
Pièce jointe : `Formulaire.pdf.exe` 4

	À lire
1	le domaine de l'expéditeur : ce n'est pas celui de l'école
2	l'urgence et la menace
3	le domaine réel du lien, en le survolant : ce n'est pas celui affiché
4	une pièce jointe exécutable

Parades contre les quatre attaques

Le mot de passe de la messagerie est à protéger en premier : il permet de réinitialiser tous les autres.

Attaque	Parade	Ce que cela demande
1. lu dans la table	l'empreinte, avec un sel et un calcul lent	rien de votre part : c'est le site qui le fait
2. deviné	long, tiré au hasard, différent sur chaque site	un gestionnaire de mots de passe, qui tire au hasard et retient
3. intercepté	le chiffrement du trajet	HTTPS, SSH, WPA sur le Wi-Fi ; déjà en place presque partout
4. pris sur le poste	un deuxième facteur	un téléphone, ou une clé physique

Le gestionnaire du navigateur ou du téléphone suffit pour commencer ; KeePassXC et Bitwarden sont les deux libres courants.

Authentification à deux facteurs

Il existe trois sortes de preuves d'identité, appelées facteurs. Un deuxième facteur ajoute au mot de passe une preuve d'une autre sorte.

Ce que je sais	Ce que je possède	Ce que je suis
un mot de passe	un téléphone, une clé physique	une empreinte digitale, un visage

Forme	Comment	Limite
code par SMS	un code reçu par message	détournable en faisant transférer le numéro
application à codes	un code changé toutes les 30 s	le code se donne sur une page d'hameçonnage
clé physique, passkey	l'appareil vérifie l'adresse du site	résiste aux deux

Les trois facteurs : ANSSI, PG-078, 2021. Comptes compromis : moins 99,2 % avec un deuxième facteur (Microsoft, 2023).

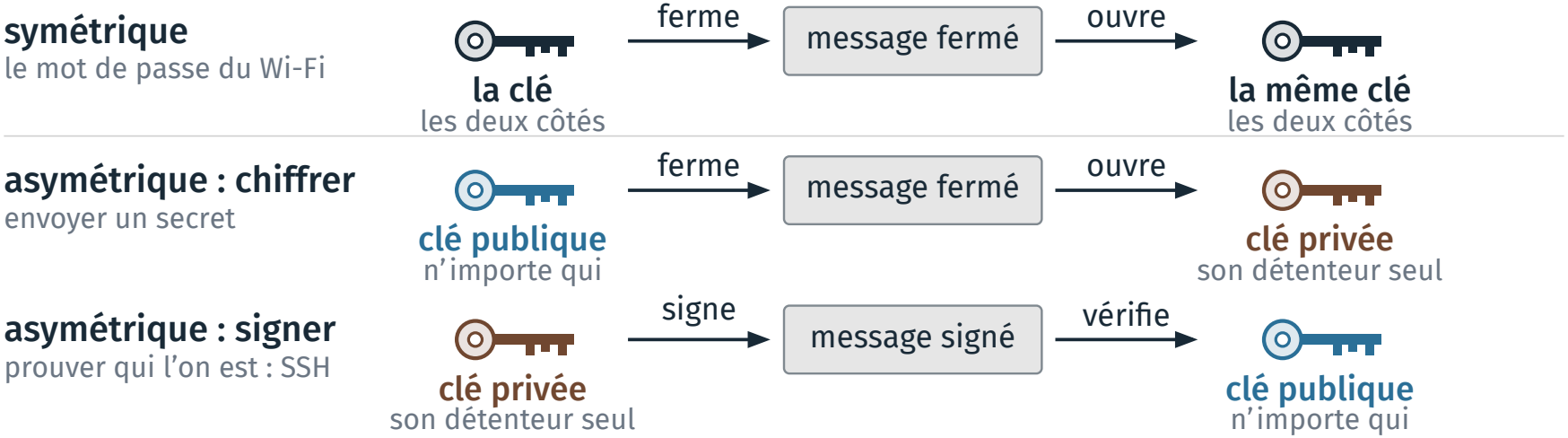
Mot de passe ou paire de clés

Pour une machine ou un programme, une paire de clés remplace le mot de passe. Rien n’est tapé, et le serveur ne garde rien de secret.

Attaque	Avec un mot de passe	Avec une paire de clés
1. lu dans la table	une empreinte, d’où un mot de passe faible se retrouve	la clé publique, inutile seule
2. deviné	30 bits pour `Marseille2024!`, 90 pour sept mots	une clé de 256 bits : hors de portée
3. intercepté	il passe sur le réseau, chiffré par HTTPS	rien de secret ne passe sur le réseau
4. pris sur le poste	il est tapé, donc exposé à l’hameçonnage	rien n’est tapé ; le fichier de la clé privée reste à protéger

Cryptographie symétrique et asymétrique

La cryptographie asymétrique emploie une paire de clés, créées ensemble. La clé publique peut être communiquée à tous ; la clé privée n'est connue que de son détenteur.



Ce que fait l'une des deux clés, seule l'autre le défait ou le vérifie. La clé privée ne se calcule pas à partir de la clé publique.

Connexion SSH avec une paire de clés

Le serveur vérifie que vous détenez la clé privée sans la recevoir.



Création d’une paire de clés SSH

`ssh-keygen` écrit les deux fichiers dans `.ssh/`, dans votre dossier personnel.

Anaconda Prompt

```
> ssh-keygen -t ed25519 -C "alice.martin@etu.ecole.fr"
Enter file in which to save the key (C:\Users\alice\.ssh\id_ed25519):
Your identification has been saved in C:\Users\alice\.ssh\id_ed25519
Your public key has been saved in C:\Users\alice\.ssh\id_ed25519.pub
The key fingerprint is:
SHA256:UdftFYrZHMVT1F9jJrEyd+9auv49Mi15wkj7o+uNUm4 alice.martin@etu.ecole.fr
```

Fichier	Taille	Contenu
`id_ed25519`	419 octets	la clé privée, lisible par vous seul ; ne jamais l’ouvrir ni la copier
`id_ed25519.pub`	107 octets	la clé publique, une seule ligne : `ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFKIay24YXuydFOmWVIyBfd3aAkOOtGncHYAqvz+ZW75 alice.martin\@etu.ecole.fr`

Accès à GitHub par token

Un token est une chaîne générée par GitHub, que git envoie par HTTPS à la place du mot de passe du compte.

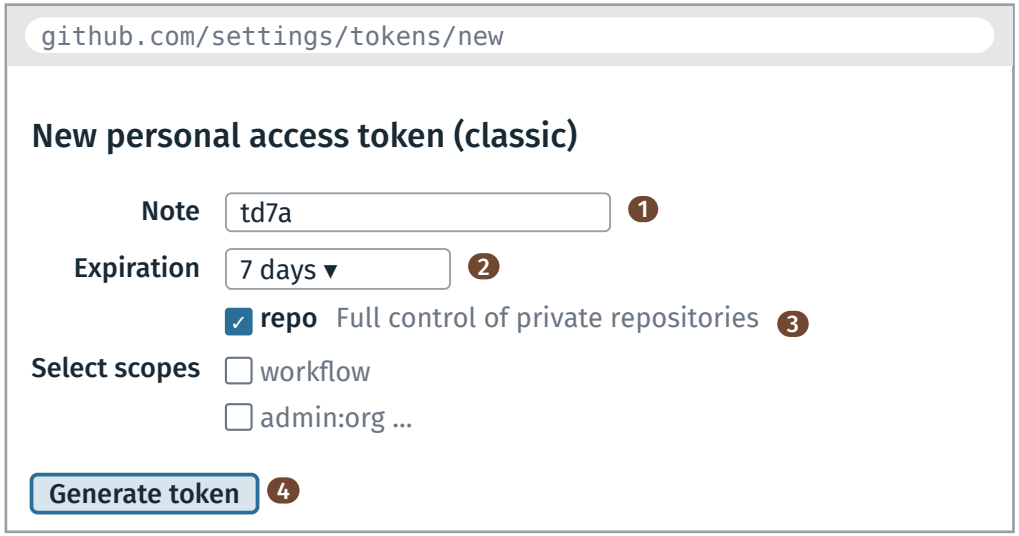


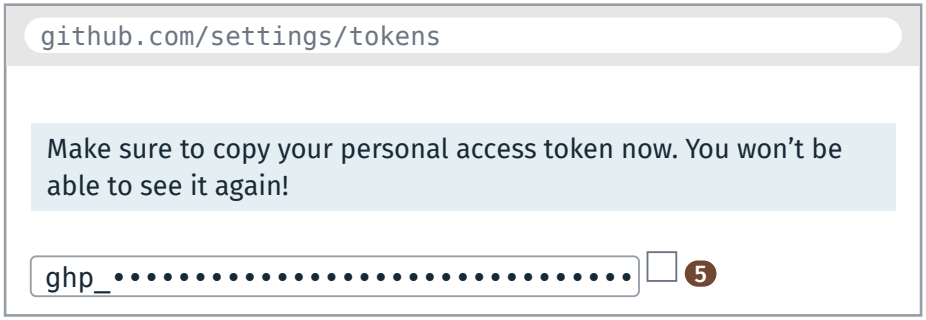
	Mot de passe du compte	Token
Créé par	vous	GitHub, au hasard
Droits	tout le compte	ceux cochés à sa création
Durée	jusqu'à son changement	jusqu'à l'expiration choisie
Après une fuite	le changer partout où il sert	supprimer ce token seul

GitHub n'affiche le token qu'une fois. Il se garde comme un mot de passe : hors de tout dépôt.

Création d’un token sur GitHub

Le token se crée dans les réglages du compte GitHub. Son nom, sa durée et ses droits se choisissent à sa création, et GitHub ne l’affiche qu’une fois.



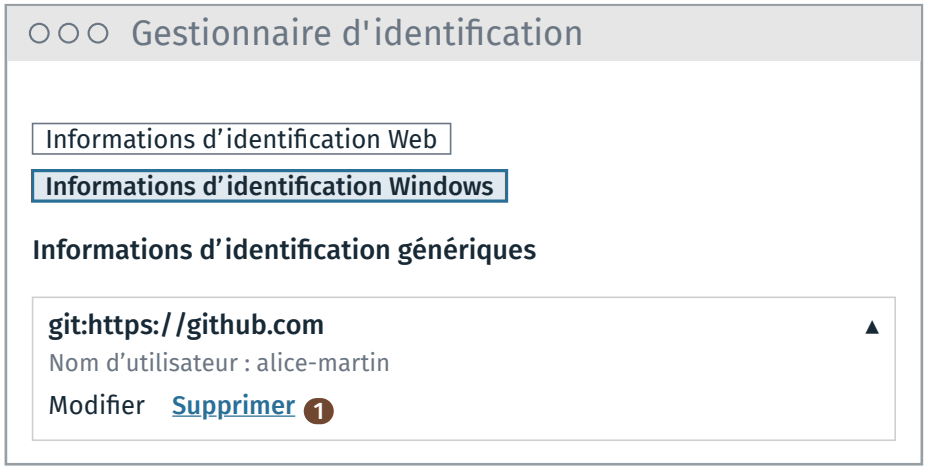


1. un nom, pour retrouver le token
2. une durée courte : 7 jours pour un TD
3. le droit d’écrire dans ses dépôts
4. la création
5. la copie, aussitôt : GitHub ne l’affiche plus

Menu du compte, Settings, Developer settings, Personal access tokens, Tokens (classic), puis Generate new token (classic).

Token sur un poste partagé

Après le premier `git push`, Windows garde le token dans son Gestionnaire d'identification, pour le compte Windows ouvert. Sur un compte Windows commun, le token reste à la disposition de l'élève suivant.



Situation	Conséquence
votre token reste sur le poste	l'élève suivant pousse dans vos dépôts
le token d'un autre est resté	votre push échoue : `Permission … denied to <autre compte>`
vous supprimez l'identifiant ❶	le push suivant redemande le compte et le token

Gestionnaire d'identification : menu Démarrer, « Gestionnaire d'identification », Informations d'identification Windows. Supprimer aussi le token sur GitHub à la fin de la séance.

Token ou clé SSH

Le token et la clé SSH remplacent tous deux le mot de passe du compte pour git.

	Token	Clé SSH
Forme	`ghp_`, puis 30 caractères tirés au hasard et 6 de contrôle	deux fichiers : la clé privée et la clé publique
Créé par	GitHub, qui l'affiche une seule fois	`ssh-keygen`, sur le poste
Envoyé à GitHub à chaque push	le token, dans la connexion chiffrée par HTTPS	une signature calculée avec la clé privée, qui reste sur le poste
Adresse du dépôt	`https://github.com/…`, port 443	`git@github.com:…`, port 22, fermé à l'école
Durée	jusqu'à l'expiration choisie	jusqu'à sa suppression sur GitHub
Droits	ceux cochés à la création	tous les dépôts du compte

Le token lui-même n'est pas chiffré. Qui le lit, dans un fichier ou sur un écran, peut s'en servir jusqu'à son expiration, comme d'un mot de passe.

TRAVAUX DIRIGÉS

TD 6a Utiliser GitHub

`cours6/`

Les secrets de vos programmes

Un jeton d’API, token en anglais, est un mot de passe utilisé par un programme. Il est exposé aux mêmes attaques qu’un mot de passe, et aussi à la publication dans un dépôt git.

Secrets et éléments publics d’un projet

Un secret donne accès à quelque chose. Il ne va ni dans un dépôt, ni dans un message, ni dans une capture d’écran.

À garder

Quoi	Donne accès à
clé privée SSH	vos comptes, vos serveurs
mot de passe	un compte
jeton d’API (*token*)	un service payant ou limité, de cartographie ou d’IA
fichier `.env`**,** `config.py`	là où sont rangés les trois précédents

À partager

Quoi	Pourquoi
clé publique SSH	ne sert à rien sans la privée
le code	le travail rendu
README, `environment.yml`	ils permettent de relancer le code
données publiques	orthophotos, BD TOPO

Un secret dans l’historique git

Le commit qui supprime le fichier ne supprime pas le commit qui l’a ajouté. `git log -p` affiche les deux.

Anaconda Prompt

```
> git log --oneline
b638a2d Supprime la clé du dépôt
9cb1911 Premier script de carte
> git log -p -- config.py
commit b638a2d  Supprime la clé du dépôt
    -CLE_API = "d7f3a9c1e5b24086"
commit 9cb1911  Premier script de carte
    +CLE_API = "d7f3a9c1e5b24086"
```

Sortie réelle, abrégée. En 2025, 28,65 millions de secrets ont été ajoutés dans des commits publics sur GitHub ; 64 % de ceux trouvés en 2022 étaient encore valides en 2026 (GitGuardian, **State of Secrets Sprawl 2026**).

Séparation du code et des secrets

Le code lit le secret dans un fichier que git ignore. Le dépôt contient un modèle de ce fichier, sans la valeur.

Dans le dépôt, pour tout le monde

```
carte.py              import config
                      cle = config.CLE_API
config.example.py     CLE_API = "à remplir"
.gitignore            config.py
README.md
```

Sur votre poste seulement

```
config.py             CLE_API = "d7f3a9c1e5b24086"
```

Qui clone le dépôt copie `config.example.py` en `config.py` et y met sa propre clé. `git status` ne liste jamais `config.py`.

Variables d’environnement : portée

Une variable créée dans un terminal reste dans ce terminal. `export` en fait une variable d’environnement, et un fichier de démarrage la pose à chaque ouverture.

Où l’écrire	Elle vaut pour
`CLE_API=abc python carte.py`	cette commande
`NOM=alice`	ce terminal seulement
`export NOM=alice`	ce terminal et ses programmes
une ligne `export` dans `~/.bash_profile`	chaque nouveau terminal

○○○ Git Bash

```
$ NOM=alice
$ bash -c 'echo "[$NOM]"'
[]
$ export NOM
$ bash -c 'echo "[$NOM]"'
[alice]
```

Dans le terminal de droite, `bash -c` lance un second programme bash. Au TD 1c, `conda init bash` a écrit dans `~/.bash_profile`.

Variables d’environnement : lecture en Python

`os.environ` donne au programme les variables qu’il a reçues. `subprocess.run` les transmet au programme lancé.

`carte.py`

```
import os
import subprocess

commande = ["bash", "-c", 'echo "bash : $CLE_API"']
subprocess.run(commande, check=True)
print("Python :", os.environ.get("CLE_API", "absente"))
```

Deux lancements

Git Bash

```
$ python carte.py
bash :
Python : absente
$ CLE_API=abc python carte.py
bash : abc
Python : abc
```

Sorties réelles. `os.environ["CLE_API"]` lève un `KeyError` si la variable est absente. Cette lecture garde un secret hors du code.

Fuite d’un secret : révocation et remplacement

Supprimer le fichier ne suffit pas. Le secret est d’abord rendu inutile, puis remplacé.



Révoquer d’abord : l’historique a peut-être déjà été copié.

Mises à jour et sauvegardes

Une mise à jour corrige une faille connue et publiée. Une sauvegarde permet de retrouver ses fichiers après la perte, le vol ou le chiffrement d'un poste.

	Mises à jour	**Sauvegardes**
Quoi	système, navigateur, Anaconda, les applications du téléphone	ce qui ne se refait pas : documents, données, photos
Quand	dès qu'elles sont proposées	régulièrement, en automatique
Règle	redémarrer quand c'est demandé	3-2-1 : trois copies, deux supports, une hors du poste
Pour le code	`conda update`, de temps en temps	un dépôt poussé sur la forge est une copie du code ; les données ignorées par git sont à sauvegarder à part

TRAVAUX DIRIGÉS

TD 6b Un secret dans l'historique — facultatif ≈ 10′

`cours6/6b_secret_historique/`

À retenir

Une forge	héberge une copie du dépôt, avec laquelle chacun se synchronise
`git clone`, `pull`, `push`	copier le dépôt, recevoir les commits, envoyer les commits
Un `pull` avant un `push`	le dépôt distant refuse un `push` s'il a des commits que le poste n'a pas
Une pull request	demande de fusionner une branche, avec les différences affichées
Un token, une clé SSH	remplacent le mot de passe du compte pour git
Un secret	jamais dans un commit ; s'il est publié, le révoquer d'abord